Software Architecture and Design Specification

Project: Console-Based Quiz Game with Multiple-Choice Questions and Scoring

Field	Details
Version	1.0
Authors	PAVAN KUMAR K M (PES2UG24CS646), ROSALIN VERMA (PES2UG24CS655), S SHREYAS KUMAR (PES2UG24CS656), VAGEESH (PES2UG24CS663)
Date	09/10/2026
Status	Draft

Revision History

Version	Date	Author	What Changed
1.0	09-10-2026	Team	First version of the design document for the quiz game

Approvals

Role	Name	Signature/Date
Course Coordinator	Pradeep Kumar	

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Online Quiz System. It presents the main components, their interactions, the selected design style, UML diagrams, interface responsibilities, error handling, security considerations, and traceability to the approved SRS requirements. The system is a C++ console client that communicates with an online server/API and data store.

1.2 Scope

The system allows students to register and log in, view and select available quizzes, retrieve MCQs and options from the online service, answer and submit quizzes, receive scores, store and view previous results, and view leaderboards. Administrators can manage quiz questions and view student results. Internet connectivity is required for normal operation.

The design covers the console-based C++ client, online server/API interactions, online data storage, and the security, performance, usability, reliability, scalability, and auditability requirements defined in the SRS. It does not specify an unrelated web or graphical client.

1.3 Who Should Read This

Developers, testers, instructors who will evaluate the project, and anyone who will maintain it later.

1.4 Short Forms Used

MCQ (Multiple Choice Question), CLI (Command Line Interface), I/O (Input/Output), Q&A (Question and Answer), RTM (Requirements Traceability Matrix), SRS (Software Requirements Specification), STP (Software Test Plan).

2. Document Overview

2.1 What This Document Gives You

It includes UML diagrams, design decisions, component responsibilities, client-server interactions, API responsibilities, error handling, security design, and traceability from the design to the SRS requirements.

2.2 Related Documents

- SRS: Software Requirements Specification – Online Quiz System v1.0
- STP: Software Test Plan
- RTM: Requirements Traceability Matrix

3. Architecture

3.1 Goals and Limits

Goals

- Normal quiz and result requests should receive a server response within 3 seconds, with the SRS acceptance target that 90% meet this threshold.
- The console client and online service should handle invalid input and communication/data errors with clear error messages rather than unhandled crashes.
- Scoring and result storage should be correct, and user credentials, quiz data, and result data should be protected.
- The C/C++ code should be neat, split into parts, and easy to maintain.

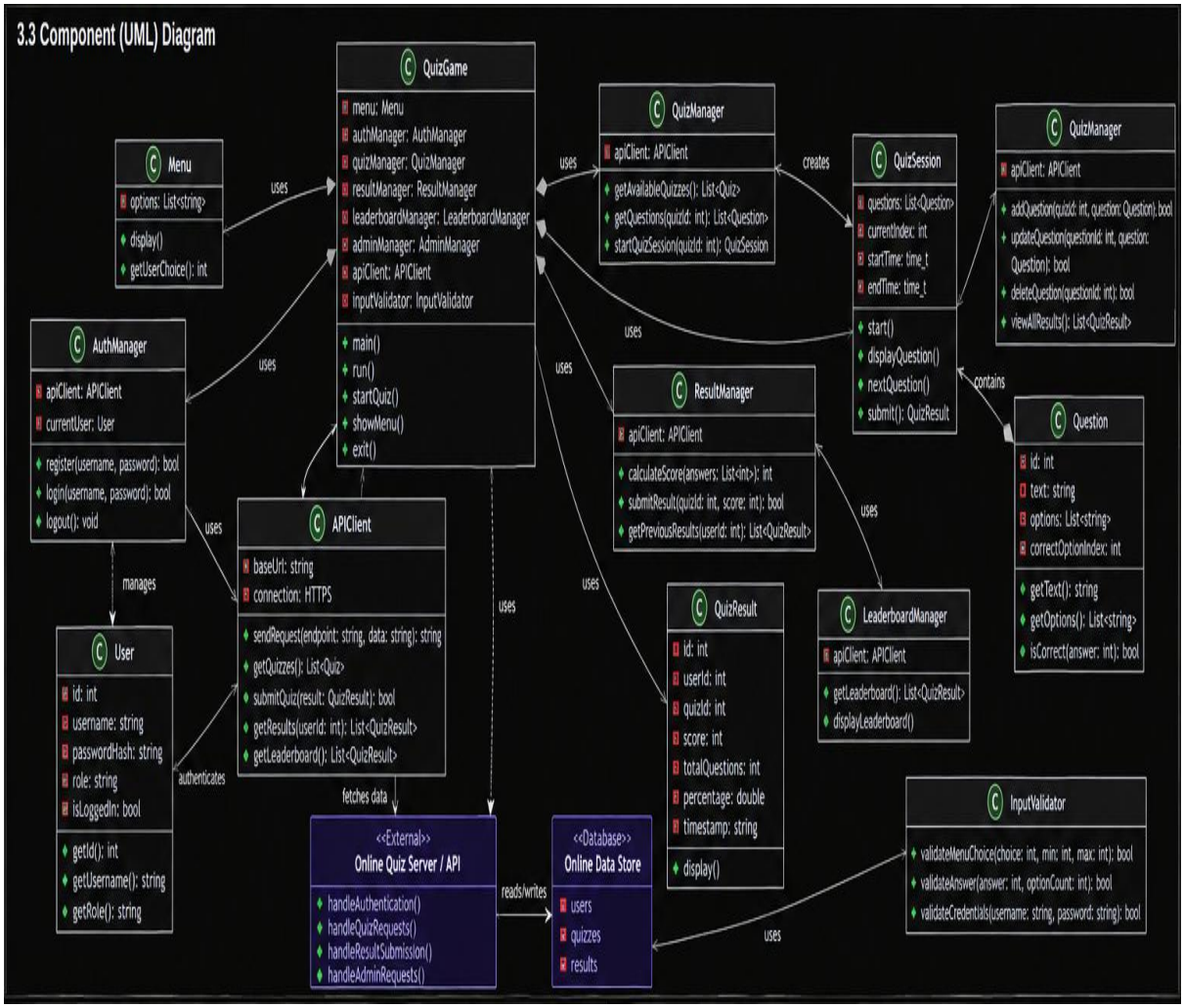
Limits

- The user-facing application is console based; normal operation requires Internet connectivity.
- The client is implemented in C++ (C++17); networking and secure client-server communication are required in addition to standard console I/O.
- The client may use a request/response interaction model; server-side concurrency must support the SRS target of at least 50 concurrent student sessions.
- The client communicates with an online server/API. An online data store holds user, quiz/question, and result information; local-only files or fixed arrays are not the system of record.
- Quiz availability and stored results are determined by the online service and data store; the client must handle unavailable services and failed requests safely.

3.2 Who Cares About What

Person or Group	What They Care About
Quiz Taker	Easy to use, clear prompts, quick response, fair scoring
Administrator/Instructor (optional)	Easy way to update questions, safe and correct data
Developers	Code that is split into clear parts and easy to maintain
Evaluators/Instructors	Easy to trace requirements, easy to test, correct results
Regulators/Auditors (academic)	No answers leaked, and user input is handled safely

3.3 Component (UML) Diagram



Classes We Plan to Use

- QuizGame – starts the C++ console client and coordinates the user workflow
- Menu – displays console menus and routes student or administrator choices
- AuthenticationClient – sends registration/login requests to the online server/API
- QuizClient – retrieves available quizzes and questions, and submits quiz answers

- QuizSession – displays one MCQ at a time, collects validated answers, and manages the attempt
- ScoreManager – calculates the result from the submitted answers and formats the summary
- ResultClient – stores results online and retrieves previous results and leaderboard data
- InputValidator – validates menu choices, credentials, and MCQ options before requests are sent
- AdminClient – sends authorized question-management requests and retrieves student results

3.4 What Each Part Does

Part	Job
QuizGame (Main Controller)	Starts the C++ console client, coordinates the menu, and routes user actions to the appropriate client module.
Menu	Shows numbered student/admin menu choices and displays clear prompts, results, and error messages.
AuthenticationClient	Sends registration and login requests to the online server/API and handles success or failure responses.
QuizClient	Requests available quizzes and selected quiz questions/options; sends completed quiz answers for submission.
QuizSession	Displays one MCQ at a time, validates selected options, and coordinates the attempt workflow.
ScoreManager	Calculates or presents the result summary using trusted quiz data and the service response.
ResultClient	Stores results through the online service and retrieves previous results and leaderboard information.
InputValidator	Validates menu choices, registration/login fields, and answer options before processing.
AdminClient	Requests authorized addition/editing/deletion of questions and retrieval of student results.

3.5 Architecture Style We Chose and Why

Our choice: a layered C++ console client connected to an online server/API and data store. The client separates presentation, application logic, and communication responsibilities; the server owns shared data and enforces authorization.

- Presentation Layer: Menu, console prompts, MCQ display, and result display.
- Application Logic Layer: QuizSession, ScoreManager, and InputValidator.
- Service/Communication Layer: AuthenticationClient, QuizClient, ResultClient, AdminClient, and the HTTPS API boundary; the online server/API accesses the data store.

Why we chose it

- Screen, logic, and data are kept separate, so each is easy to understand.
- Each part can be tested on its own.

- It is simple to build in C/C++ without any framework.
- The client-server design is required by the SRS. A modular client and service/data separation keep responsibilities testable without changing the console-based user interface.

3.6 Tools and Data Storage

Item	What We Use
Language	C++ (C++17) for the console client
Compiler	GCC / MinGW / MSVC
Input and Output	Standard text-based console
File Handling	Used only where appropriate for client configuration or diagnostics; not as the primary store for shared quiz/results data
Data Storage	Online data store accessed through the online server/API for users, quizzes, questions, and results
Build	Makefile or g++ command line
Platform	Windows or Linux terminal with Internet connectivity

3.7 Risks and How We Handle Them

Risk	What We Will Do
Invalid user input or malformed request	Validate client inputs and server-side request data; return clear errors without crashing.
Online server/API unavailable or times out	Handle network errors and timeouts safely; show a clear message and avoid claiming an operation succeeded.
Unauthorized access to user, quiz, or result data	Use HTTPS/TLS, secure password hashing, authenticated sessions, and server-side authorization.
Answer key or credentials exposed	Keep credentials out of logs; do not reveal correct answers before submission; protect data in transit.
Result submitted for another student	Server validates that every quiz submission belongs to the authenticated student.
Audit events missing or sensitive data logged	Timestamp login attempts, quiz submissions, and administrator changes; never log passwords.

3.8 Linking Requirements to Design

Requirement (SRS Ref)	Design Part That Handles It
OQS-F-001 Registration	AuthenticationClient, Server/API, User Data Store
OQS-F-002 Student login	AuthenticationClient, Server/API, User Data Store

Requirement (SRS Ref)	Design Part That Handles It
OQS-F-003 Available quizzes	QuizClient, Server/API, Quiz Data Store
OQS-F-004 Select quiz	Menu, QuizClient
OQS-F-005 Retrieve questions/options	QuizClient, Server/API, Question Data Store
OQS-F-006 Display one MCQ at a time	QuizSession, Menu
OQS-F-007 Validate answer option	InputValidator, QuizSession
OQS-F-008 Navigate questions	QuizSession, QuizClient
OQS-F-009 Submit completed quiz	QuizClient, Server/API
OQS-F-010 Calculate score	Server scoring logic, ScoreManager
OQS-F-011 Display final result summary	ScoreManager, Menu
OQS-F-012 Store result online	ResultClient, Server/API, Result Data Store
OQS-F-013 View previous results	ResultClient, Server/API, Result Data Store
OQS-F-014 Display leaderboard	ResultClient, Server/API, Result Data Store
OQS-F-015 Admin question/result management	AdminClient, Server/API, Authorization, Data Store
OQS-NF-001 Response time: 90% within 3 seconds	Server/API, QuizClient, ResultClient
OQS-NF-002 Clear menus/prompts/errors	Menu, InputValidator, API error handling
OQS-NF-003 99% monthly availability target	Online service deployment/operations
OQS-NF-004 At least 50 concurrent sessions	Server/API and data-store capacity
OQS-NF-005 Timestamped audit events	Server audit logging
OQS-SR-001 HTTPS/TLS	Client-server communication layer
OQS-SR-002 Secure password hashing; no plaintext	Authentication service and user data store
OQS-SR-003 Authenticated administrator authorization	Authentication/authorization service, AdminClient
OQS-SR-004 Submission belongs to authenticated student	Server submission validation
OQS-SR-005 Log failed logins/admin actions without passwords	Server audit logging

3.9 Security Design

Threat Check (STRIDE)

STRIDE is a checklist of six common types of security threats. Here is how we handle each one.

Threat	How We Handle It
Spoofing	Require student authentication and authenticated administrator accounts; enforce roles on the server.
Tampering	Validate requests and submissions server-side; protect communication with HTTPS/TLS and store results through the service.
Repudiation	Timestamp login attempts, quiz submissions, and administrator changes in audit logs.
Information Disclosure	Use secure password hashing, never log passwords, and do not reveal answer keys before submission.

Threat	How We Handle It
Denial of Service	Validate inputs, handle timeouts and malformed requests, and test service response/concurrency targets.
Elevation of Privilege	Enforce administrator authorization on the server; hiding an admin menu in the client alone is insufficient.

Security Goals

- Quiz integrity — only the program's logic can calculate scores and results.
- Answer-key secrecy — correct answers are not shown before the user submits.

4. Design

4.1 Design Overview

The Online Quiz System is a console-based C++ client-server application. Its main interaction steps are:

- The client starts and displays the console menu.
- The student or administrator selects an action; the client validates inputs and sends the appropriate request to the online server/API.
- For a quiz attempt, the service provides available quizzes and questions; QuizSession displays each MCQ and collects the student's answer.
- The client submits the completed attempt; scoring/result processing returns a result summary.
- The server stores the result in the online data store, while the client can request previous results and leaderboard information.

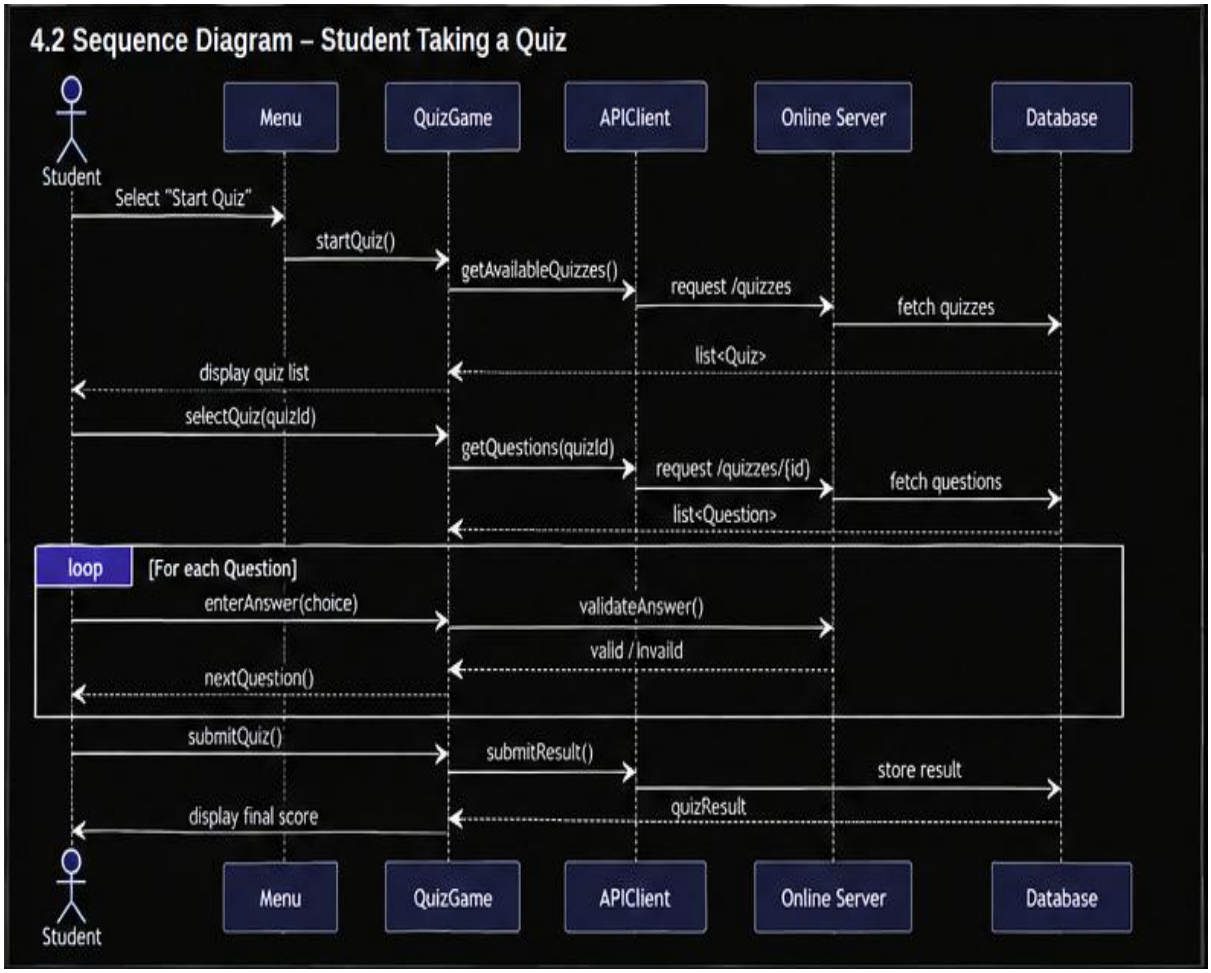
Client-server requests use request/response messages over HTTPS/TLS. The server validates authentication and authorization and checks that a submission belongs to the authenticated student.

4.2 UML Sequence Diagrams

Flow 1: Start the Quiz and Answer Questions

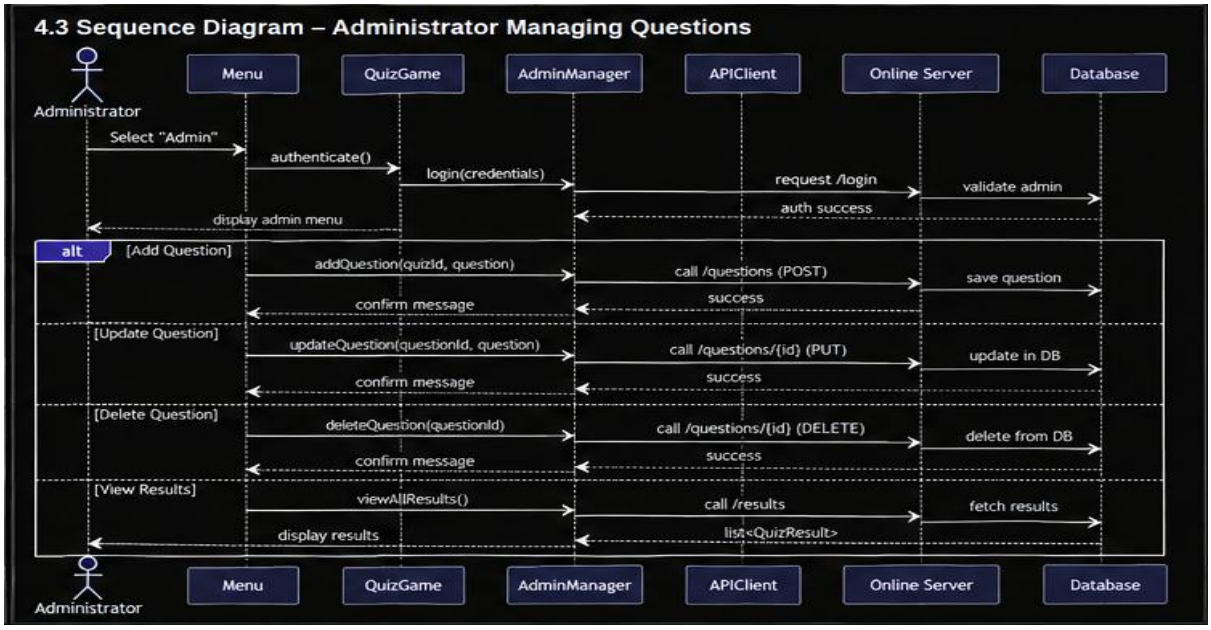
```
Student -> Console Client: Register / log in
Console Client -> Server/API: HTTPS registration or login request
Server/API -> Data Store: Validate credentials / create account
Data Store --> Server/API: Account or authentication result
Server/API --> Console Client: Success or error response
Student -> Console Client: Select quiz
Console Client -> Server/API: Request available quizzes and questions
    QuizSession -> User: display question + options
    User -> QuizSession: enter option
    QuizSession -> InputValidator: validate(option)
    InputValidator --> QuizSession: valid / invalid
    QuizSession -> ScoreManager: updateScore(correct?)
```

Console Client -> Server/API: Submit answers over HTTPS
Server/API -> Scoring Service: Validate student and calculate result
Scoring Service -> Data Store: Store quiz result
Server/API --> Console Client: Final score and result summary



Flow 2: Administration and Results Retrieval

Administrator -> Console Client: Log in
Console Client -> Server/API: Send admin authentication request over HTTPS
Server/API -> Data Store: Verify credentials and administrator role
Data Store --> Server/API: Authentication/authorization result
Server/API --> Console Client: Access granted or denied
Administrator -> Console Client: Add/edit/delete questions or view student results



Console Client -> Server/API: Send authorized administrator request
Server/API -> Data Store: Update question data or read student results
Data Store --> Server/API: Operation result / result records
Server/API --> Console Client: Confirmation or error / result data
Console Client -> Administrator: Display status or student results
Student -> Console Client: Request previous results / leaderboard
Console Client -> Server/API: Request previous results / leaderboard
Server/API -> Data Store: Read authorized results and ranking data
Data Store --> Server/API: Result records
Server/API --> Console Client: Previous results / leaderboard
Console Client -> Student: Display requested data

4.3 API Design

This section describes the main client-side operations and the online API interactions. Exact endpoint paths and payload schemas are implementation details to be agreed by the team; the responsibilities below follow the SRS.

Part 1: QuizClient and AuthenticationClient

Function: requestAvailableQuizzes() -> QuizList
Input: authenticated session and quiz/request parameters
Output: registration/login status, available quizzes, or selected quiz questions/options
Errors: AUTHENTICATION_FAILED, NETWORK_ERROR, SERVER_ERROR, INVALID_RESPONSE

Part 2: Scoring and Quiz Submission

Function: calculateScore(submittedAnswers, answerKey) -> QuizResult
Input: submitted answers and trusted quiz answer data
Output: correct, incorrect, total questions, and percentage
Errors: INVALID_SUBMISSION, INVALID_QUIZ_DATA

Function: submitQuiz(quizId, submittedAnswers, authenticatedSession) -> ResultResponse

Input: quiz identifier, submitted answers, authenticated session
Output: final result summary returned by the online service
Errors: AUTHENTICATION_FAILED, UNAUTHORIZED_SUBMISSION, NETWORK_ERROR, SERVER_ERROR

Part 3: ResultClient and AdminClient

Function: sendRequest(endpoint, request, authenticatedSession) -> Response
Function: storeResult / retrieveResults / manageQuestions through the server API
Errors: NETWORK_ERROR, AUTHORIZATION_FAILED, INVALID_RESPONSE, STORAGE_ERROR

4.4 Errors, Logs, and Monitoring

The client validates user input and handles invalid credentials, rejected requests, unavailable server/API, timeouts, invalid responses, and failed result storage with clear messages. Sensitive failures must not expose credentials or internal details.

Audit events include login attempts, quiz submissions, and administrator changes, with timestamps. Passwords must never be recorded in logs. The service uses HTTPS/TLS, secure password hashing, role-based administrator authorization, and submission ownership validation as required by the SRS.

Monitoring and audit review should use the recorded events and the test/operational checks available for the project; do not treat a local optional log file as a replacement for the SRS auditability requirement.

4.5 User Experience

- Clear, numbered menu options.
- Simple, easy-to-read prompts with the same style everywhere.
- Numbered answer options for each question (1–4).
- Display the final score, correct answers, total questions, and percentage after submission, as specified in OQS-F-011.
- Readable text, no meaning shown by colour alone, and text that works with screen readers.
- Use clear status and error messages for network failures, invalid input, and rejected requests; do not introduce an optional timer as a required feature unless the SRS is updated.

4.6 Implementation Considerations and Next Steps

- Implement and verify registration and login against the online service (OQS-F-001, OQS-F-002).
- Implement available-quiz listing, quiz selection, and online question retrieval (OQS-F-003 to OQS-F-005).
- Implement MCQ display, answer validation, question navigation, submission, scoring, and result display (OQS-F-006 to OQS-F-011).
- Implement online result storage, previous-result viewing, leaderboard display, and administrator question/result management (OQS-F-012 to OQS-F-015).
- Implement and verify the SRS non-functional targets: 3-second response target, clear console prompts/errors, 99% monthly service availability target, 50 concurrent sessions, and timestamped audit events.

- Implement and verify security requirements for HTTPS/TLS, hashed passwords, administrator authorization, submission ownership validation, and password-free audit logs.
- Develop unit, integration, system, regression, performance, usability, reliability, and security tests consistent with the Software Test Plan.

5. Appendices

5.1 Glossary

Term	Meaning
MCQ	Multiple Choice Question
CLI	Command Line Interface
SRS	Software Requirements Specification
RTM	Requirements Traceability Matrix
STRIDE	A security checklist: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
ADR	Architecture Decision Record

5.2 References

- IEEE 42010 – Architecture Description
- OWASP – Secure Coding Practices
- NIST SP 800-160 – Systems Security Engineering
- SRS: Online Quiz System v1.0

5.3 Tools

- PlantUML / draw.io – for drawing UML diagrams
- g++ / GCC / MSVC – for compiling the code
- Makefile – for automating the build
- Git – for version control